

vnr-qckit

Claude Code plugin cho VNR Playwright QC automation.

Mục lục

- [Cài đặt](#)
 - [Cấu trúc plugin](#)
 - [Tổng quan kiến trúc](#)
 - [Các skill và lệnh](#)
 - [Luồng thực thi theo giai đoạn](#)
 - [Chi tiết từng skill](#)
 - [Giai đoạn 1 — Cài đặt & Khởi tạo](#)
 - [qc_pre](#)
 - [qc_setup](#)
 - [qc_basepage](#)
 - [Giai đoạn 2 — Khám phá Ứng dụng](#)
 - [qc_detect_component](#)
 - [qc_component_rule](#)
 - [qc_user_flow](#)
 - [Giai đoạn 3 — Mapping Page Object](#)
 - [qc_map_flow](#)
 - [qc_url_page_map](#)
 - [qc_sub-system-map](#)
 - [Giai đoạn 4 — Sinh Test Case](#)
 - [qc_generate](#)
 - [Giai đoạn 5 — Chạy Test Tự động](#)
 - [qc_auto_test](#)
 - [Giai đoạn 6 — Phân loại Kết quả](#)
 - [qc_triage](#)
 - [Bảng Input/Output nhanh](#)
 - [Cấu trúc thư mục output](#)
 - [Yêu cầu](#)
 - [Quy ước](#)
-

Cài đặt

Bước 1 — Thêm marketplace

```
claude plugin marketplace add <đường-dẫn-hoặc-repo>
```

Ví dụ từ local path:

```
claude plugin marketplace add D:/SourceCode/AutomationProcess/vnr-qckit
```

Ví dụ từ GitHub:

```
claude plugin marketplace add your-org/vnr-qckit
```

Bước 2 — Install plugin

```
claude plugin install vnr-qckit@vnr-qckit --scope project
```

`--scope project` → ghi vào `.claude/settings.json` của project, shared với cả team.

Bước 3 — Reload trong Claude Code

```
/reload-plugins
```

Cấu trúc plugin

```
vnr-qckit/
├── .claude-plugin/
│   ├── plugin.json           ← Manifest: tên, version, mô tả plugin
│   └── marketplace.json      ← Catalog để self-host marketplace
├── .claude/
│   └── settings.local.json   ← Permissions, hooks cục bộ
├── .mcp.json                 ← Khai báo MCP server (playwright)
├── bin/
│   └── vnr-qckit.js          ← CLI entry point (npm bin)
├── hooks/
│   └── qckit-permissions.js  ← Hook tự động cấp quyền tool
├── skills/                   ← Mỗi thư mục con = 1 slash command
│   ├── playwright_cli/      ← /vnr-qckit:playwright_cli
│   │   ├── SKILL.md         ← Driver browser qua MCP (internal)
│   │   └── references/      ← Tài liệu tham chiếu Playwright
│   │       ├── request-mocking.md
│   │       ├── running-code.md
│   │       ├── session-management.md
│   │       ├── store-state.md
│   │       ├── test-generation.md
│   │       └── tracing.md
│   ├── qc_pre/              ← /vnr-qckit:qc_pre (orchestrator)
│   │   └── SKILL.md
│   ├── qc_setup/            ← /vnr-qckit:qc_setup (internal)
│   │   └── SKILL.md
│   ├── qc_basepage/         ← /vnr-qckit:qc_basepage (internal)
│   │   └── SKILL.md
│   ├── qc_detect_component/ ← /vnr-qckit:qc_detect_component
│   │   └── SKILL.md
│   ├── qc_component_rule/   ← /vnr-qckit:qc_component_rule
│   │   └── SKILL.md
│   ├── qc_user_flow/        ← /vnr-qckit:qc_user_flow
│   │   └── SKILL.md
│   └── qc_map_flow/         ← /vnr-qckit:qc_map_flow
│       └── SKILL.md
```

└─ qc_url_page_map/ └─ SKILL.md	← /vnr-qckit:qc_url_page_map
└─ qc_sub-system-map/ └─ SKILL.md	← /vnr-qckit:qc_sub-system-map
└─ qc_generate/ └─ SKILL.md	← /vnr-qckit:qc_generate
└─ qc_auto_test/ └─ SKILL.md	← /vnr-qckit:qc_auto_test
└─ qc_triage/ └─ SKILL.md	← /vnr-qckit:qc_triage
└─ templates/ └─ basepage_template.md	← Template BasePage.ts (hardcode)
└─ template-component-rule.md	← Template format component-rule.md
└─ prompts/ └─ components.md	← Danh sách Angular component + selector hints
└─ components_core.md	← DOM HTML reference cho Kendo/legacy
└─ create_catalog.md	← Prompt sinh data catalog
└─ helper/ └─ playwright_cli.md	← Hướng dẫn dùng playwright-cli MCP
└─ package.json	← npm metadata (bin, files, engines)
└─ INSTALL.md	← Hướng dẫn cài đặt chi tiết
└─ README.md	← Tài liệu này
└─ VNR-QCKIT-FLOW.md	← Tài liệu luồng đầy đủ (format, ví dụ)

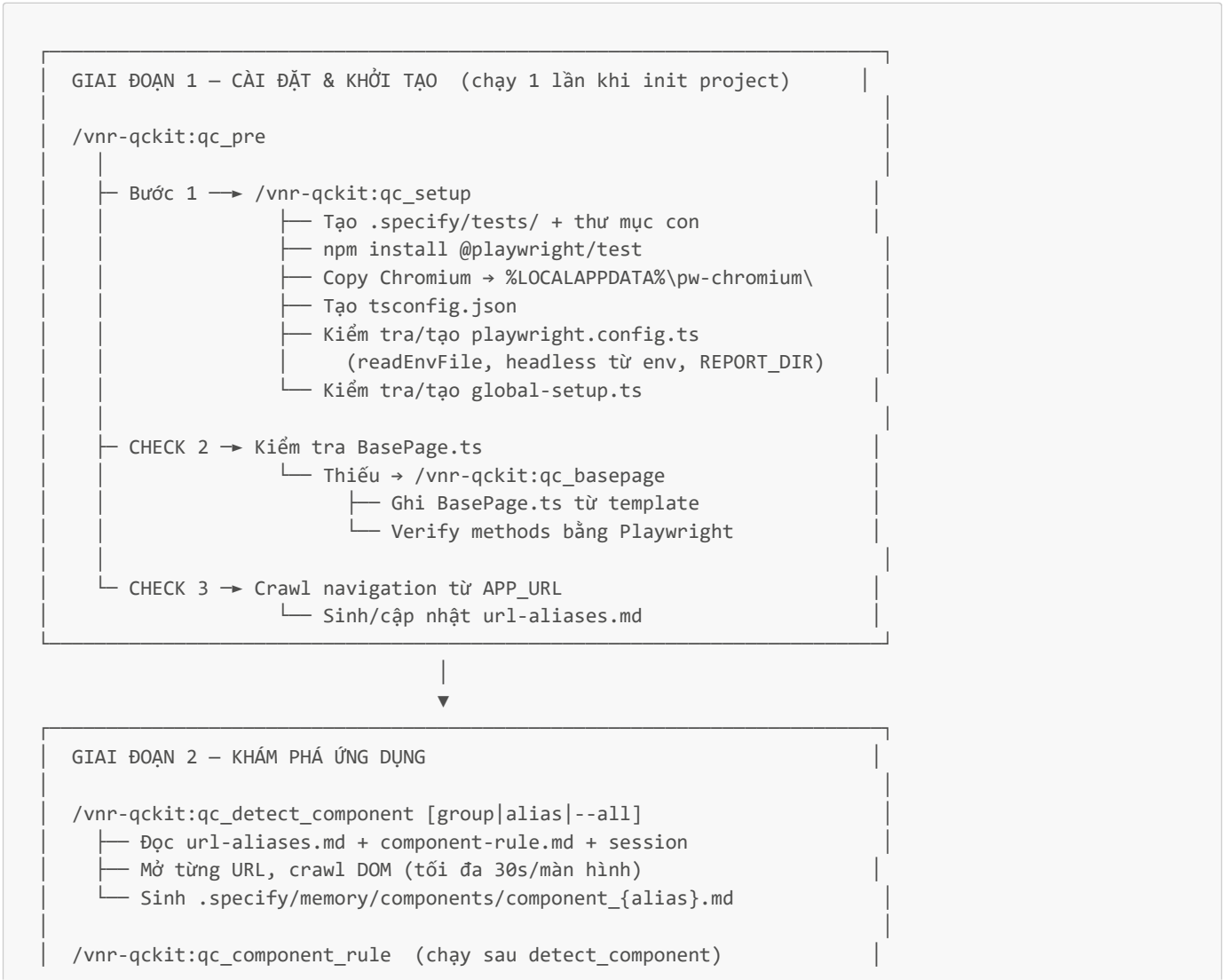
Tổng quan kiến trúc

<u>GĐ 1 – CÀI ĐẶT</u>	<u>GĐ 2 – KHÁM PHÁ</u>	<u>GĐ 3 – MAPPING</u>
qc_pre └─ qc_setup └─ qc_basepage └─ url-aliases.md	qc_detect_component └─ crawl DOM/component └─ component_{alias}.md qc_component_rule └─ component-rule.md qc_user_flow └─ userflow-{group}.md	qc_map_flow └─ {AliasPage}.ts └─ {AliasLocator}.ts └─ locator-map.md qc_url_page_map └─ url-page-map.md qc_sub-system-map └─ sub-system-map.json
<u>GĐ 4 – SINH TEST CASE</u>	<u>GĐ 5 – CHẠY TEST</u>	<u>GĐ 6 – PHÂN LOẠI</u>
qc_generate └─ testcase.md └─ datafake.json └─ knowledge/*.md └─ _IMPACT_INDEX.json	qc_auto_test └─ lọc TC Auto chưa PASSED └─ data chaining (context) └─ sinh spec.ts có cấu trúc └─ playwright-cli (execution) └─ playwright-mcp (discovery) └─ cập nhật Mục 4 + Locator.ts └─ HISTORY.md	qc_triage └─ PASS / APP_BUG / INFRA └─ Severity table └─ triage-*.md

Các skill và lệnh

Lệnh	Loại	Mô tả ngắn
/vnr-qckit:qc_pre	Orchestrator	Setup toàn bộ project: Playwright, BasePage, url-aliases.md
/vnr-qckit:qc_setup	Internal	Cài Playwright + Chromium, tạo playwright.config.ts, global-setup.ts
/vnr-qckit:qc_basepage	Internal	Tạo/cập nhật BasePage.ts từ template chuẩn
/vnr-qckit:qc_detect_component	Khám phá	Crawl DOM từng màn hình → sinh component_{alias}.md
/vnr-qckit:qc_component_rule	Khám phá	Tổng hợp selector rule → sinh/cập nhật component-rule.md
/vnr-qckit:qc_user_flow	Khám phá	Crawl action từng màn hình → sinh userflow-{group}.md
/vnr-qckit:qc_map_flow	Mapping	Sinh {AliasPage}.ts + {AliasLocator}.ts + locator-map.md
/vnr-qckit:qc_url_page_map	Mapping	Khớp alias → Page Object đã có → sinh url-page-map.md
/vnr-qckit:qc_sub-system-map	Mapping	Trích xuất màn hình → group → sinh sub-system-map.json
/vnr-qckit:qc_generate	Sinh test	Sinh testcase.md, datafake.json, knowledge/*.md từ spec PBI
/vnr-qckit:qc_auto_test	Chạy test	Chạy TC Auto chưa PASSED, sinh spec.ts có cấu trúc, data chaining, cập nhật kết quả
/vnr-qckit:qc_triage	Phân tích	Phân loại PASS/APP_BUG/INFRA, sinh bảng Severity
/vnr-qckit:playwright_cli	Driver	Điều khiển browser qua MCP (dùng bởi các skill khác)

Luồng thực thi theo giai đoạn



- Đọc tất cả component_{alias}.md
- Tổng hợp component types mới chưa có rule
- Append vào .specify/rules/component-rule.md

/vnr-qckit:qc_user_flow [alias|group|--all]

- Crawl action + form fields trên từng màn hình (60s/màn hình)
- Sinh ASCII flow diagram: Group→Alias→Feature→Screen→Action
- Ghi .specify/memory/userflow-{group}.md



GIAI ĐOẠN 3 – MAPPING PAGE OBJECT

/vnr-qckit:qc_map_flow [alias|--all|--pbi PBI_ID]

- Đọc component_temp_{alias}.md + component-rule.md + userflow
- Sinh .specify/tests/pages/{group}/{AliasName}Locator.ts
- Sinh .specify/tests/pages/{group}/{alias}/{AliasName}Page.ts
- Sinh .specify/tests/knowledge/{group}/{alias}/locator-map.md

/vnr-qckit:qc_url_page_map

- Glob *Page.ts + *Locator.ts trong pages/
- Khớp alias → Page class theo score >= 0.6
- Sinh .specify/tests/pages/url-page-map.md

/vnr-qckit:qc_sub-system-map

- Parse bảng "Đã có Page Object" trong url-page-map.md
- Sinh .specify/memory/sub-system-map.json



GIAI ĐOẠN 4 – SINH TEST CASE (per PBI)

/vnr-qckit:qc_generate <PBI_ID>

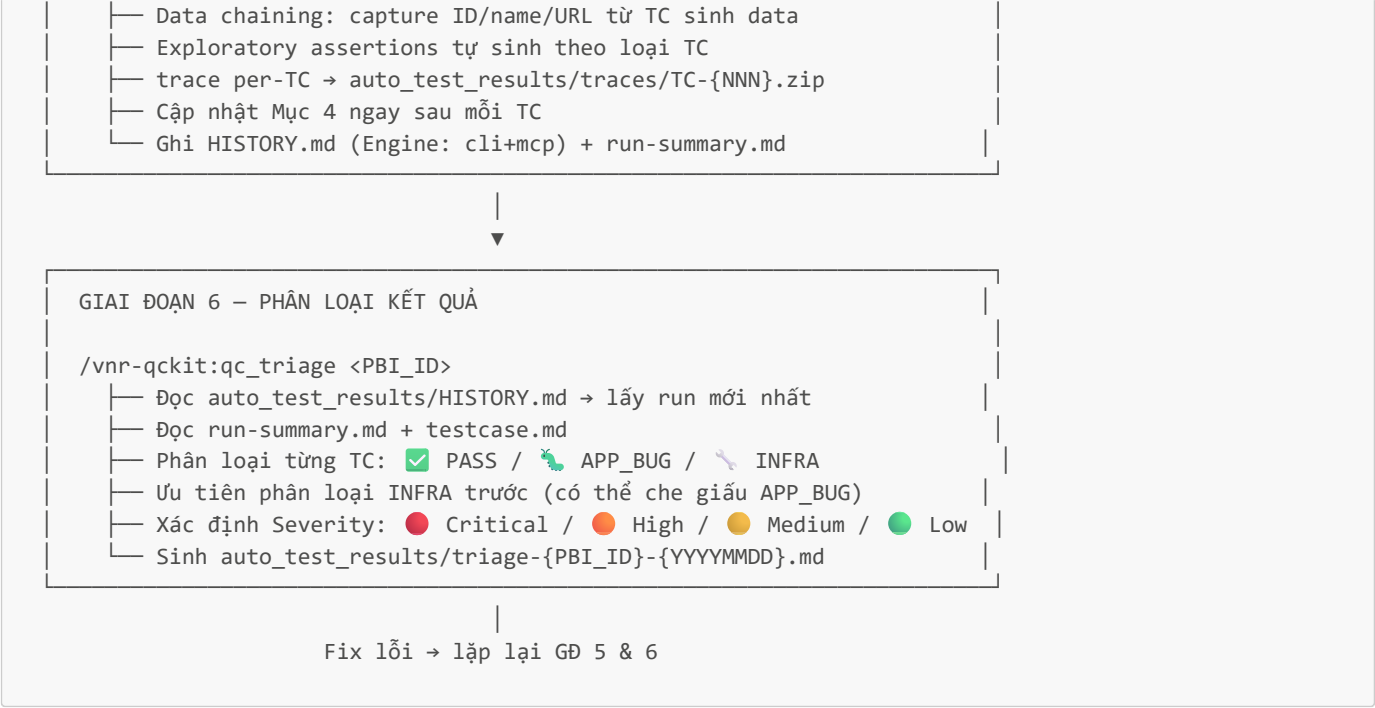
- Đọc song song: spec.md, plan.md, userflow, domain-knowledge, mockup ảnh, testcase_template.md (nếu có)
- Phân tích: màn hình, fields, business rules, nhóm TC, --uutien
- Sinh .specify/specs/{PBI_ID}/testcase.md
(TC ID: template → {PBI_ID}_{GROUP}_NNN, fallback → TC-NNN)
- Sinh .specify/tests/playwright/{PBI_ID}/datafake.json
- Sinh .specify/tests/knowledge/{PBI_ID}/screen-summary.md
+ field-catalog.md
+ _IMPACT_INDEX.json (tạo/cập nhật luôn)



GIAI ĐOẠN 5 – CHẠY TEST TỰ ĐỘNG (per PBI, lặp lại)

/vnr-qckit:qc_auto_test <PBI_ID>

- Đọc .env → testcase.md → sub-system-map.json
- Bóc tách Locator.ts + Page.ts per-màn hình
- Phân tích chuỗi phụ thuộc data giữa TC
- Cập nhật cột ★ trong Mục 4
- Load session + login check + save session
- Sinh playwright/{PBI_ID}/{PBI_ID}.spec.ts (structured)
(import Locator/Page via baseUrl, runtimeContext, describe/test)
- Sinh playwright/{PBI_ID}/{TC_ID}.spec.ts (per-TC)
- Discovery [MISSING_SELECTOR] qua mcp browser_snapshot
→ cập nhật component_{alias}.md
→ cập nhật Locator.ts + Page.ts
- Thực thi --uutien → p0 → p1 → p2...
playwright-cli (execution), mcp (discovery + Kendo combobox)



Lưuồng xử lý:

```

Bước 1 → Gọi /vnr-qckit:qc_setup (chờ hoàn thành)
CHECK 2 → Kiểm tra BasePage.ts; thiếu → gọi /vnr-qckit:qc_basepage
CHECK 3 → Đọc APP_URL từ .env → crawl navigation → ghi url-aliases.md

```

Báo cáo kết quả:

```

[v] qc_pre hoàn thành
Playwright + Chromium : [v] đã cài
playwright.config.ts   : OK | [edit] đã sửa | [new] đã tạo
global-setup.ts        : OK | [new] đã tạo
BasePage.ts            : EXISTS | CREATED | UPDATED | FAILED
url-aliases.md          : added={n}, updated={n}, unchanged={n}, warning={n}

```

Skill: qc_setup (Internal)

Được gọi tự động bởi **qc_pre**. Có thể chạy độc lập để reset cấu hình.

Gọi bằng	/vnr-qckit:qc_setup hoặc tự động từ qc_pre
Arguments	(không có)
Mục đích	Cài Playwright + Chromium, kiểm tra/tạo playwright.config.ts và global-setup.ts

Input:

File/Thư mục	Bắt buộc	Ghi chú
.specify/tests/.env	✓	Phải tồn tại trước khi chạy
{PLUGIN_DIR}\tools\pw-chromium\chrome-win\	✓	Chromium bundled sẵn trong plugin

Output:

File	Trạng thái	Ghi chú
.specify/tests/ (thư mục + node_modules)	Tạo + cài	@playwright/test được npm install
.specify/tests/tsconfig.json	Tạo nếu thiếu	ES2020, commonjs, strict: true
.specify/tests/playwright.config.ts	Tạo/sửa	headless từ env, readEnvFile, REPORT_DIR, executablePath
.specify/tests/global-setup.ts	Tạo nếu thiếu	Session login (tự reuse/refresh user.json)
%LOCALAPPDATA%\pw-chromium\chrome-win\	Copy	Chromium cố định, dùng làm executablePath

Lưuồng xử lý (6 CHECK tuần tự):

```

CHECK 0 → Tạo .specify/tests/ + runs/ + pages/
          npm install @playwright/test --prefix .specify/tests
          Copy Chromium → %LOCALAPPDATA%\pw-chromium\chrome-win\
          Tạo tsconfig.json nếu thiếu

CHECK 1 → playwright.config.ts tồn tại?
          MISSING → tạo mới với chuẩn (readEnvFile, headless, executablePath, REPORT_DIR)
          OK       → tiếp CHECK 2

```

```

CHECK 2 → readEnvFile có trong config?
MISSING → chèn hàm readEnvFile + đọc baseUrl từ .env
OK → tiếp CHECK 3

CHECK 3 → headless đọc từ env (PW_HEADLESS)?
MISSING → sửa thành: (process.env.PW_HEADLESS ?? envVars.PW_HEADLESS ?? 'true') !==
'false'
OK → tiếp CHECK 4

CHECK 4 → REPORT_DIR có trong config?
MISSING → thêm biến reportDir = process.env.REPORT_DIR || timestamp
OK → tiếp CHECK 5

CHECK 5 → global-setup.ts tồn tại?
MISSING → ghi từ template chuẩn
OK → báo cáo kết quả

CHECK 6 → BasePage.navigate() dùng _resolveAppUrl()?
MISSING/OUTDATED → edit BasePage.ts: thêm _resolveAppUrl(), sửa navigate()
OK → báo cáo kết quả

```

Skill: qc_basepage (Internal)

Được gọi tự động bởi qc_pre. Có thể chạy độc lập khi cần cập nhật BasePage.

Gọi bằng /vnr-qckit:qc_basepage hoặc tự động từ qc_pre

Arguments (không có)

Mục đích Tạo/cập nhật BasePage.ts — lớp cha chứa common methods cho toàn bộ Page Object

Input:

File	Bắt buộc	Ghi chú
.specify/tests/pages/BasePage.ts	—	Đọc nếu tồn tại để kiểm tra trước khi ghi đè
.specify/tests/.env + .env.{ENV}	✓	Lấy APP_URL để verify methods

Output:

File	Trạng thái	Ghi chú
.specify/tests/pages/BasePage.ts	Tạo mới / giữ nguyên	Template chuẩn với _resolveAppUrl()
.specify/tests/tmp/_basepage_verify.spec.ts	Tạm (tự xóa sau)	Test tạm để verify từng method

Methods trong BasePage.ts:

```

navigate(url)           — resolve _APP_URL + hash routing, goto + domcontentloaded
handleConfirmDialog(btn) — click confirm dialog nếu xuất hiện, no-op nếu không có
waitForToast(text)      — chờ toast/notification chứa text
verifyUrl(pattern)      — assert URL khớp string/RegExp
inputTextbox(loc, val)   — click + fill textbox
verifyTextbox(loc, exp)  — expect(locator).toHaveValue(expected)
inputCombobox(loc, val)  — fill + chờ dropdown jQuery autocomplete + click item
inputCheckbox(loc, checked) — tích/bỏ tích nếu trạng thái hiện tại khác yêu cầu
verifyCheckbox(loc, exp) — expect checked/not.toBeChecked()
inputTable(tbody, addRow, rows, columnMap) — nhập dữ liệu theo danh sách TableRow[]

```

Luồng xử lý (5 Bước):

```
Bước 1 → Kiểm tra BasePage.ts đã tồn tại?  
      False → tạo từ template chuẩn  
      True  → đọc, không ghi đè nếu không có yêu cầu sửa  
  
Bước 2 → Ghi BasePage.ts từ template (khi thiếu)  
  
Bước 3 → Tạo test tạm .specify/tests/tmp/_basepage_verify.spec.ts  
      → chạy: npx playwright test tmp/_basepage_verify.spec.ts --workers=1  
  
Bước 4 → Method fail? Chỉ được sửa: selector, timeout, logic nội bộ  
      KHÔNG được sửa: tên method, số tham số, kiểu trả về, signature  
  
Bước 5 → Xóa file test tạm → báo cáo: Pass / Skip / FailFix / Failed
```

Giai đoạn 2 — Khám phá Ứng dụng

Skill: qc_detect_component

Gọi bằng	/vnr-qckit:qc_detect_component [group-code alias1 alias2 --all]
Mục đích	Crawl DOM từng màn hình, phát hiện component theo rule và element chưa có rule
Tools	mcp__playwright__* (ưu tiên) → playwright-cli → npx playwright (fallback)

Input:

File	Bắt buộc	Ghi chú
.specify/tests/url-aliases.md	✓	Danh sách alias + path cần crawl
.specify/rules/component-rule.md	✓	Selector rule cho từng component type
.specify/tests/user.json	✓	Session login; tự refresh nếu hết hạn
.specify/tests/.env + .env.{ENV}	✓	APP_URL, AUTH_USERNAME, AUTH_PASSWORD

Output:

File	Trạng thái	Ghi chú
.specify/memory/components/component_{alias}.md	Tạo mới / ghi đè	Một file per màn hình

Format output component_{alias}.md:

```
# Component List – {alias}  
> URL: {path} | Sinh: {ISO_DATE}  
  
## {ComponentType} – {mô tả} ({n} fields)  
| # | Label | formControlName | Placeholder | Required | Context |  
| ---|---|---|---|---|---|  
| 1 | Tên phiếu | tenPhieu | Nhập tên... | ✓ | dialog |
```

Luồng xử lý:

```
BƯỚC 0 → Đọc .env → APP_URL; kiểm tra url-aliases.md + component-rule.md
        Nạp session user.json vào playwright-mcp
        Parse argument → danh sách alias cần xử lý

BƯỚC 1 → Với từng alias (tối đa 30s/màn hình):
        - Mở URL → wait networkidle
        - Detect component tags theo rule trong component-rule.md
        - Thu thập: label, FormControlName, placeholder, required, context
        - Ghi component_{alias}.md

BƯỚC 2 → Báo cáo: tổng màn hình xử lý, component tìm thấy, component mới
```

Skill: qc_component_rule

Chạy sau qc_detect_component khi phát hiện component type mới.

Gọi bằng	/vnr-qckit:qc_component_rule
Arguments	(không có)
Mục đích	Tổng hợp component types mới chưa có rule, cập nhật component-rule.md

Input:

File	Bắt buộc	Ghi chú
.specify/memory/components/component_*.md	✓	Ít nhất 1 file; thiếu → DỪNG
.specify/rules/component-rule.md	—	Đọc nếu tồn tại để biết type nào đã có rule
{PLUGIN_DIR}/templates/template-component-rule.md	—	Template format output
{PLUGIN_DIR}/prompts/components.md	—	DOM HTML reference Angular components
{PLUGIN_DIR}/prompts/components_core.md	—	DOM HTML reference Kendo/legacy

Output:

File	Trạng thái	Ghi chú
.specify/rules/component-rule.md	Tạo mới / append	Định nghĩa container, label, input selector cho mỗi type

Luồng xử lý:

```
BƯỚC 0 → Quét .specify/memory/components/component_*.md
        Không tìm thấy file nào → DỪNG

BƯỚC 1 → Đọc đồng thời: component-rule.md (file đích) + reference DOM files + template

BƯỚC 2 → Tổng hợp unique component types từ tất cả component_*.md
        → Lọc ra các type chưa có trong component-rule.md
        → Ánh xạ mỗi type → DOM HTML từ components.md / components_core.md

BƯỚC 3 → Với type đã có rule: hỏi user ghi đè hay giữ nguyên

BƯỚC 4 → Append entry mới vào component-rule.md theo format chuẩn

BƯỚC 5 → Báo cáo: CREATED / UPDATED / SKIPPED / TODO
```

Skill: `qc_user_flow`

Gọi bằng	<code>/vnr-qckit:qc_user_flow [alias1 alias2 group-code --all]</code>
Mục đích	Crawl user flow thực tế, sinh ASCII diagram, ghi userflow-{group}.md
Tools	<code>mcp__playwright__*</code> (ưu tiên) → <code>playwright-cli</code> → <code>npx playwright</code> (fallback)

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/url-aliases.md</code>	✓	Danh sách alias + path cần crawl
<code>.specify/tests/user.json</code>	✓	Session login hợp lệ
<code>.specify/tests/.env</code> + <code>.env.{ENV}</code>	✓	APP_URL
<code>.specify/memory/flow-index.md</code>	—	Cập nhật entry nếu tồn tại

Output:

File	Trạng thái	Ghi chú
<code>.specify/memory/userflow-{group}.md</code>	Tạo / cập nhật	ASCII flow diagram + mô tả màn hình
<code>.specify/memory/flow-index.md</code>	Cập nhật	Index thời gian + trạng thái URL
<code>.specify/tests/url-aliases.md</code>	Cập nhật nếu phát hiện URL mới	Thêm alias <code>[new]</code>

Format output `userflow-{group}.md`:

```
Group → Alias → Feature → Screen → Action
ATT
  att_leave_day – DS Ngày nghỉ
  Toolbar: [Thêm mới: open-dialog] [Xuất Excel: export] [Xóa: delete]
  Happy path – Tạo mới:
    1. Navigate att_leave_day
    2. Click Thêm mới → dialog mở
    3. inputCombobox(loaiNgayNghỉ, "...")
    4. Click Lưu → toast "Thêm mới thành công"
```

Luồng xử lý:

```
BƯỚC 0 → Kiểm tra env, url-aliases.md, session
          Parse argument → danh sách alias

BƯỚC 1 → Crawl từng màn hình (tối đa 60s/màn hình):
  - Mở URL, chụp snapshot (mcp__playwright__browser_snapshot)
  - Phân loại: List / Form / Detail / Tab / Dashboard
  - Crawl toolbar actions (label, type, state, disabled?)
  - Crawl form fields (label, type, required, validation, options)
  - Crawl sub-tabs nếu có
  - Crawl pagination, filter, search

BƯỚC 2 → Chuẩn hoá:
  - Suy luận happy path, luồng phụ
  - Map field_type → BasePage method
  - Sinh ASCII flow diagram

BƯỚC 3 → Ghi userflow-{group}.md
```

BƯỚC 4 → Cập nhật url-aliases.md (nếu phát hiện URL mới)

BƯỚC 5 → Cập nhật flow-index.md

Giai đoạn 3 — Mapping Page Object

Skill: qc_map_flow

Gọi bằng /vnr-qckit:qc_map_flow [alias1 alias2 | --all | --pbi <PBI_ID>]

Mục đích Sinh {AliasName}Locator.ts + {AliasName}Page.ts từ component_temp + userflow

Input:

File	Bắt buộc	Ghi chú
vnr-qckit/detected_components/component_temp_{alias}.md	✓	Thiếu → DỪNG, nhắc chạy qc_detect_component
.specify/tests/url-aliases.md	✓	Alias → path + group detection
.specify/rules/component-rule.md	Ưu tiên	Selector rules; fallback sang template
.specify/memory/userflow-{group}.md	Ưu tiên	Tìm trước
.specify/memory/userflow.md	Fallback	Dùng khi không có file theo group
.specify/specs/{PBI_ID}/testcase.md	Tùy chọn	Suy luận method cần ưu tiên (--pbi)

Output:

File	Trạng thái	Ghi chú
.specify/tests/pages/{group}/{AliasName}Locator.ts	Tạo / ghi đè	Khai báo tất cả locator
.specify/tests/pages/{group}/{alias}/{AliasName}Page.ts	Tạo / ghi đè	Action methods, import Locator
.specify/tests/knowledge/{group}/{alias}/locator-map.md	Tạo / ghi đè	Tài liệu tham chiếu locator

Quy tắc đặt tên:

alias: att_leave_day (group: att) → AttLeaveDayLocator.ts + AttLeaveDayPage.ts

alias: cat_leave_day_type (group: cat) → CatLeaveDayTypeLocator.ts + CatLeaveDayTypePage.ts

Ưu tiên locator strategy:

1. getByRole('{role}', { name: '{label}' })

2. getByLabel('{label}')

3. locator(container).filter(hasText).locator(input)

4. locator('[formcontrolname="..."]')

5. locator(':nth-child(n) input')
- ARIA role (nút, link)

– label rõ ràng

– custom component

– formControlName

– fallback index

Luồng xử lý:

BƯỚC 0 → Xác định alias list + group theo argument

Kiểm tra file nguồn; thiếu → DỪNG

BƯỚC 1 → Đọc song song: url-aliases.md, component-rule.md, userflow file
 Xây: ComponentRoleMap + FieldInventory + ActionMap

BƯỚC 2 → Tính locator strategy tối ưu cho từng field/button

BƯỚC 3 → Sinh {AliasName}Locator.ts (khai báo locators)
 Sinh {AliasName}Page.ts (action methods, import Locator)

BƯỚC 4 → Sinh locator-map.md tài liệu tham chiếu

BƯỚC 5 → Cập nhật _IMPACT_INDEX.json + flow-index.md nếu có

Skill: qc_url_page_map

Gọi bằng	/vnr-qckit:qc_url_page_map
Arguments	(không có)
Mục đích	Quét Page Object đã có, khớp alias → Page class, sinh url-page-map.md

Input:

File	Bắt buộc	Ghi chú
.specify/tests/url-aliases.md	✓	Danh sách alias + path
.specify/tests/pages/**/*Page.ts	—	Các Page Object đã sinh
.specify/tests/pages/**/*Locator.ts	—	Các Locator đã sinh

Output:

File	Trạng thái	Ghi chú
.specify/tests/pages/url-page-map.md	Tạo / ghi đè	Bảng: đã khớp + chưa khớp + thống kê

Thuật toán khớp (score ≥ 0.6):

alias: att_leave_day → segments normalize → "attleaveday"
 Page class: AttLeaveDayPage → normalize → "attleaveday"
 Score = ký tự khớp liên tiếp / độ dài chuỗi ngắn hơn

Luồng xử lý:

BƯỚC 0 → Kiểm tra url-aliases.md + thư mục pages/
 BƯỚC 1 → Glob *Page.ts + *Locator.ts → extract ClassName + path
 Ghép cặp Page ↔ Locator theo prefix
 BƯỚC 2 → Parse url-aliases.md → alias + path + section
 BƯỚC 3 → Tính score khớp mỗi alias × Page class; chọn score cao nhất ≥ 0.6
 BƯỚC 4 → Sinh url-page-map.md (đã khớp / chưa khớp / thống kê theo section)
 BƯỚC 5 → Báo cáo: tổng alias, đã khớp, chưa có PO

Skill: qc_sub-system-map

Gọi bằng	<code>/vnr-qckit:qc_sub-system-map</code>
Arguments	<i>(không có)</i>
Mục đích	Trích xuất mapping màn hình → group, sinh sub-system-map.json dùng cho qc_generate

Input:

File	Bắt buộc	Ghi chú
<code>.specify/tests/pages/url-page-map.md</code>	✓	Chỉ đọc section "Đã có Page Object"

Output:

File	Trạng thái	Ghi chú
<code>.specify/memory/sub-system-map.json</code>	Tạo / ghi đè	JSON: [{ ma_man_hinh, ten_man_hinh, group }]

Quy tắc trích xuất group và ma_man_hinh:

```

Page File: att/att_leave_day/AttLeaveDayPage.ts
→ group      = "att"          (segment[0])
→ ma_man_hinh = "att_leave_day" (segment[1])

Page File: hre/HreProfilePage.ts
→ group      = "hre"
→ ma_man_hinh = "HreProfile"   (fallback: tên file bỏ suffix Page)

```

Luồng xử lý:

```

BƯỚC 0 → Kiểm tra url-page-map.md; thiếu → DỪNG
BƯỚC 1 → Parse bảng "Đã có Page Object"
BƯỚC 2 → Trích xuất group + ma_man_hinh từ cột Page File
BƯỚC 3 → Deduplicate theo page_file (giữ dòng đầu)
BƯỚC 4 → Ghi sub-system-map.json (sort: group ASC, ma_man_hinh ASC)
BƯỚC 5 → Báo cáo số lượng

```

Giai đoạn 4 — Sinh Test Case**Skill: qc_generate**

Gọi bằng	<code>/vnr-qckit:qc_generate <PBI_ID></code>
Arguments	<code>PBI_ID</code> — bắt buộc
Mục đích	Sinh testcase.md, datafake.json, knowledge/*.md từ spec + plan + flow

Input:

File	Bắt buộc	Ghi chú
<code>.specify/specs/{PBI_ID}/spec.md</code>	✓	Thiếu → DỪNG ngay
<code>.specify/specs/{PBI_ID}/plan.md</code>	✓	Thiếu → DỪNG ngay
<code>.specify/memory/userflow-{group}.md</code>	✓	Thiếu → DỪNG, hỏi group màn hình

File	Bắt buộc	Ghi chú
<code>.specify/templates/testcase_template.md</code>	—	Template chuẩn — ưu tiên dùng khi có
<code>.specify/memory/constitution.md</code>	—	Quy tắc dự án
<code>.specify/memory/domain-knowledge.md</code>	—	Business domain
<code>.specify/specs/{PBI_ID}/assets/*.png,*.jpg</code>	—	UI mockup (đọc qua vision)
<code>.specify/tests/data-catalog/categories/</code>	—	Dữ liệu thực (vendors, customers, ...)

Output:

File	Trạng thái	Ghi chú
<code>.specify/specs/{PBI_ID}/testcase.md</code>	Tạo mới	Danh sách TC đầy đủ; ID format phụ thuộc template
<code>.specify/tests/playwright/{PBI_ID}/datafake.json</code>	Tạo mới	Data giả có nghĩa nghiệp vụ
<code>.specify/tests/knowledge/{PBI_ID}/screen-summary.md</code>	Tạo mới	Mô tả màn hình, luồng, business rules
<code>.specify/tests/knowledge/{PBI_ID}/field-catalog.md</code>	Tạo mới	Danh mục field và validation
<code>.specify/tests/knowledge/_IMPACT_INDEX.json</code>	Tạo mới / cập nhật	Luôn tạo/cập nhật — không cần tạo thủ công

TC ID format:

```

Khi có template : {PBI_ID}_{GROUP}_{FEATURE_CODE}_{NNN}   (ví dụ: 15552_CAT_CHUYENKHO_001)
Khi không có    : TC-NNN   (fallback)
```

Nhóm test case sinh ra:

```

P1 – Luồng chính (happy path): CRUD cơ bản
P1 – Validation: required fields, format, boundary
P1 – Business rule: logic nghiệp vụ đặc thù
P2 – Edge case: empty state, error response
P2 – Permission: phân quyền theo role
```

Phân loại Loại TC:

```

✅ Auto   – Toàn bộ steps qua Web browser, verify qua DOM, message lỗi đã xác định
👉 Manual – Thiết bị vật lý, visual check, message [TODO], data setup phức tạp trong DB
```

Mặc định là Auto — chỉ chuyển Manual khi có lý do cụ thể. Mỗi TC phải có **Lý do loại**.

Khái niệm --uutien:

```

TC thiết yếu nhất – nếu fail thì PBI không thể chấp nhận
Mỗi PBI: 3-7 TC ưu tiên, không quá 40% tổng TC
qc_auto_test chạy --uutien trước; nếu fail → dừng toàn bộ batch
```

Luồng xử lý (6 Bước):

Bước 1 → Đọc song song tất cả tài liệu

Bước 2 → Phân tích spec: màn hình, fields, business rules, nhóm TC; đánh dấu --uutien

Bước 3 → Sinh testcase.md (dùng template nếu có; fallback TC-NNN)

Bước 4 → Sinh datafake.json → .specify/tests/playwright/{PBI_ID}/
Quy tắc: KHÔNG dùng foo/bar/test123; ưu tiên data-catalog → domain → format VN

Bước 5 → Sinh knowledge/*.md (screen-summary, field-catalog)

Bước 6 → Tạo/cập nhật _IMPACT_INDEX.json (luôn thực hiện)

Giai đoạn 5 — Chạy Test Tự động

Skill: qc_auto_test

Gọi bằng	/vnr-qckit:qc_auto_test <PBI_ID>
Arguments	<PBI_ID> (bắt buộc) / --reset TC-001,TC-002 / --reset-all / --uutien TC-001
Mục đích	Chạy TC Auto chưa PASSED, sinh spec.ts có cấu trúc (import Locator/Page), thực thi, data chaining giữa TC, cập nhật kết quả vào testcase.md
Tools	playwright-cli (execution) + playwright-mcp (discovery DOM khi cần)

Phân công tool:

playwright-cli → MỌI thao tác thực thi:
navigate, fill, click, check, assert_visible/text/url/value/count/hidden
evaluate (auth guard, capture ID/URL/text từ DOM)
screenshot, load/save_storage_state, tracing_start/stop

playwright-mcp → CHỈ dùng cho 2 trường hợp:
browser_snapshot (discovery selector component mới chưa có trong Locator.ts)
Kendo combobox sequence: browser_click + browser_type + browser_snapshot +
browser_click

Input (đọc theo thứ tự, không gom song song):

File	Bắt buộc	Thứ tự đọc
.specify/tests/.env + .env.{ENV}	✓	1 — lấy APP_URL
.specify/specs/{PBI_ID}/testcase.md	✓	2 — lọc TC, cập nhật cột ★
.specify/memory/sub-system-map.json	✓	3 — lấy URL + Locator/Page path
{AliasName}Locator.ts + {AliasName}Page.ts	✓	4 — per-màn hình
.specify/tests/user.json	✓	5 — trước login check
.specify/tests/playwright/{PBI_ID}/datafake.json	✓	6 — trước khi sinh spec.ts

Output:

File	Trạng thái	Ghi chú
.specify/specs/{PBI_ID}/testcase.md	Cập nhật	Cột Trạng thái + cột ★ trong Mục 4
.specify/tests/playwright/{PBI_ID}/{PBI_ID}.spec.ts	Tạo mới	Tất cả TC trong 1 file (chạy batch)

File	Trạng thái	Ghi chú
<code>.specify/tests/playwright/{PBI_ID}/{TC_ID}.spec.ts</code>	Tạo mới	1 file / 1 TC (rerun đơn lẻ)
<code>.specify/specs/{PBI_ID}/auto_test_results/traces/TC-{NNN}.zip</code>	Tạo mới	Trace per-TC
<code>.specify/specs/{PBI_ID}/auto_test_results/screenshots/TC-{NNN}-fail-*.png</code>	Khi FAILED	Ảnh chụp khi thất bại / auth expired
<code>.specify/specs/{PBI_ID}/auto_test_results/run-{DATETIME}-summary.md</code>	Tạo mới	Tóm tắt lần chạy + data chaining + component mới
<code>.specify/specs/{PBI_ID}/auto_test_results/HISTORY.md</code>	Append	Lịch sử chạy (Engine: cli+mcp; cột New components)
<code>.specify/memory/components/component_{alias}.md</code>	Cập nhật nếu có component mới	Tag [NEW - /qc_auto_test {PBI_ID}]
<code>{AliasName}Locator.ts / {AliasName}Page.ts</code>	Cập nhật nếu cần	Thêm selector/method mới

Cấu trúc spec.ts sinh ra:

```
// {PBI_ID}.spec.ts – sinh bởi /qc_auto_test
import { CatLeaveDayTypePage } from 'pages/cat/cat_leave_day_type/CatLeaveDayTypePage'
import datafake from './datafake.json'

// Runtime context – data sinh từ TC trước dùng cho TC sau
const runtimeContext = { createdId: undefined, createdName: undefined, selectedValues: {} }

test.describe('★ --uutien', () => {
  test.beforeEach(async ({ page }) => { await checkAuthGuard(page) })
  test('[TC-001] @auto @uutien – Tên TC', async ({ page }) => { ... })
})
test.describe('p0', () => { ... })
test.describe('p1', () => { ... })
```

Data chaining giữa TC:

TC sinh data (tạo/sửa/xóa) → capture ID/name/URL vào runtimeContext ngay sau PASSED
 TC dùng data từ TC trước → đọc runtimeContext trước, fallback về datafake nếu chưa có

Quy tắc quan trọng:

TC PASSED / FAILED → KHÔNG chạy lại (trừ --reset)
 TC SKIPPED / Chưa test → chạy lại lần tiếp
 Chỉ chạy TC Loại = Auto; bỏ qua Manual
 Thứ tự ưu tiên: --uutien → p0 → p1 → p2 → ...
 TC --uutien FAILED → dừng toàn bộ batch
 Login check bắt buộc → PASSED phải save_storage_state ngay; FAILED → dừng
 playwright.config.ts phải có testDir: './playwright'

Luồng xử lý (8 Bước):

Bước 0 → Đọc .env → testcase.md (lọc, sort, cập nhật ★) → sub-system-map.json
 Đọc Locator.ts + Page.ts per-màn hình → bóc tách selector + method

```
Phân tích chuỗi phụ thuộc data giữa TC (runtime_context analysis)
Bước 1 → Load user.json → playwright-cli load_storage_state → login check
        PASSED → save_storage_state ngay
Bước 2 → Đọc datafake.json → sinh spec.ts có cấu trúc
        Discovery [MISSING_SELECTOR] qua browser_snapshot (mcp)
        Cập nhật component_{alias}.md + Locator.ts + Page.ts nếu có component mới
Bước 3 → Thực thi từng TC tuần tự (playwright-cli chính):
        A. tracing_start per-TC
        B. Auth guard check: evaluate "window.location.href"
        C. Inject runtime context → thực thi bước
        D. Assertion chính + Exploratory assertions (tự sinh theo loại TC)
        E. Capture runtime context nếu TC sinh data (sau PASSED)
        F. tracing_stop → traces/TC-{NNN}.zip
        G. Cập nhật Mục 4 ngay → kiểm tra điều kiện dừng
Bước 4 → Xác định trạng thái cuối từng TC
Bước 5 → Ghi run-{DATETIME}-summary.md (Data Chaining log + Exploratory Warnings + Component mới)
Bước 6 → Append HISTORY.md (Engine: cli+mcp; cột New components)
Bước 7 → Cập nhật bảng Tóm tắt trong testcase.md
```

Giai đoạn 6 — Phân loại Kết quả

Skill: qc_triage

Gọi bằng	/vnr-qckit:qc_triage <PBI_ID>
Arguments	PBI_ID — bắt buộc
Mục đích	Phân loại kết quả PASS/APP_BUG/INFRA, xác định Severity, sinh báo cáo triage

Input:

File	Bắt buộc	Ghi chú
.specify/specs/{PBI_ID}/auto_test_results/HISTORY.md	✓	Thiếu → DỪNG (nhắc chạy qc_auto_test trước)
.specify/specs/{PBI_ID}/auto_test_results/run-{DATETIME}-summary.md	Ưu tiên	Summary của run mới nhất
.specify/specs/{PBI_ID}/testcase.md	✓	Lấy mô tả, nhóm, priority, kết quả inline

Output:

File	Trạng thái	Ghi chú
.specify/specs/{PBI_ID}/auto_test_results/triage-{PBI_ID}-{YYYYMMDD}.md	Tạo / ghi đè	Báo cáo phân loại đầy đủ + hành động tiếp theo

3 loại phân loại:

Loại	Ký hiệu	Định nghĩa
PASS	✓	Test exit 0, tất cả assertion green
APP_BUG	🐛	App sai nghiệp vụ: feature missing, validation sai, business rule sai, UI sai spec
INFRA	🔧	Lỗi môi trường: session expired, host mismatch, selector stale, network timeout, script bug

Bảng quyết định phân loại:

```

TC FAILED →
├── TimeoutError + redirect/login/cookie mismatch → INFRA (session)
├── TimeoutError + selector không match + FE chưa deploy → APP_BUG (feature missing)
├── TimeoutError + selector stale sau FE refactor → INFRA (stale selector)
├── Assertion fail + app không block khi phải block → APP_BUG (validation missing)
├── Assertion fail + message sai → APP_BUG (wrong message)
├── TypeError / undefined trong script → INFRA (script bug)
├── networkidle timeout + server không phản hồi → INFRA (network/env)
└── Không rõ nguyên nhân → APP_BUG (cần dev xác nhận)

TC PASSED →  PASS

```

Severity cho APP_BUG:

Severity	Điều kiện
 Critical	Feature core P1 không hoạt động, hoặc security/permission bị vi phạm
 High	Business rule P1 sai, validation bắt buộc bị thiếu, filter hiển thị data sai
 Medium	Message/toast sai nội dung, field không persist, conditional show/hide sai
 Low	UI cosmetic, label sai, format ngày sai, default value sai (không ảnh hưởng save)

Lưuồng xử lý (5 Bước):

Bước 1 → Kiểm tra HISTORY.md → xác định RUN_ID và run mới nhất
Đọc summary file + testcase.md

Bước 2 → Phân loại từng TC: PASS / APP_BUG / INFRA
Ưu tiên loại trừ INFRA trước khi kết luận APP_BUG

Bước 3 → Xác định Severity cho mỗi APP_BUG
Gom bug ảnh hưởng nhiều TC vào 1 entry

Bước 4 → Sinh triage-{PBI_ID}-{YYYYMMDD}.md:
 - Bảng tổng quan (PASS/APP_BUG/INFRA + %)
 - Danh sách PASS
 - Bảng APP_BUG theo Severity + chi tiết từng bug
 - Bảng INFRA + action fix
 - Hành động tiếp theo ưu tiên (INFRA fix trước)

Bước 5 → Báo cáo console: tổng quan + bug nổi bật + bước tiếp theo

Bảng Input/Output nhanh

Skill	Input chính	Output chính	Ghi đề?
qc_setup	.env, plugin Chromium	playwright.config.ts, global-setup.ts, tsconfig.json	Tạo/sửa
qc_basepage	template hardcode	BasePage.ts	Tạo nếu thiếu
qc_pre	.env, app đang chạy	url-aliases.md + gọi qc_setup + qc_basepage	Cập nhật

Skill	Input chính	Output chính	Ghi đề?
qc_detect_component	url-aliases.md, component-rule.md, session	components/component_{alias}.md	Ghi đề
qc_component_rule	component_*.md, DOM reference	component-rule.md	Append
qc_user_flow	url-aliases.md, session	userflow-{group}.md, flow-index.md	Cập nhật
qc_map_flow	component_temp_{alias}.md, component-rule.md, userflow-{group}.md	{AliasLocator}.ts, {AliasPage}.ts, locator-map.md	Ghi đề
qc_url_page_map	url-aliases.md, *Page.ts, *Locator.ts	url-page-map.md	Ghi đề
qc_sub-system-map	url-page-map.md	sub-system-map.json	Ghi đề
qc_generate	spec.md, plan.md (cả hai bắt buộc), userflow-{group}.md	testcase.md, playwright/{PBI_ID}/datafake.json, knowledge/*.md, _IMPACT_INDEX.json	Tạo mới
qc_auto_test	testcase.md, playwright/{PBI_ID}/datafake.json, sub-system-map.json, Locator.ts/Page.ts	playwright/{PBI_ID}/{PBI_ID}.spec.ts + per-TC spec, traces/, screenshots/, HISTORY.md; cập nhật Mục 4 + Locator.ts/Page.ts nếu có component mới	Cập nhật/Append
qc_triage	HISTORY.md, run-summary.md, testcase.md	trriage-{PBI_ID}-{YYYYMMDD}.md	Ghi đề (cùng ngày)

Cấu trúc thư mục output

.specify/ - tests/ .env .env.local - user.json - playwright.config.ts - global-setup.ts - tsconfig.json - url-aliases.md - playwright/ {PBI_ID}/ {PBI_ID}.spec.ts {TC_ID}.spec.ts - datafake.json - pages/ - BasePage.ts - url-page-map.md {group}/ {AliasName}Locator.ts {alias}/ {AliasName}Page.ts - data-catalog/ - categories/ - knowledge/ _IMPACT_INDEX.json {PBI_ID}/ - screen-summary.md - field-catalog.md {group}/{alias}/ - locator-map.md	← INPUT: ENV=local ← INPUT: APP_URL=..., PW_HEADLESS=false ← Session state (sinh bởi global-setup.ts) ← [qc_setup] testDir: './playwright' ← [qc_setup] login + lưu user.json ← [qc_setup] baseUrl: "" ← [qc_pre] alias → path từ crawl navigation ← testDir – Playwright chạy từ đây ← [qc_auto_test] tắt cả TC (batch) ← [qc_auto_test] per-TC (rerun đơn) ← [qc_generate] dữ liệu test ← [qc_basepage] lớp cha common methods ← [qc_url_page_map] alias → Page Object ← [qc_map_flow] khai báo locators ← [qc_map_flow] action methods ← INPUT cho qc_generate ← [qc_generate] tạo/cập nhật per PBI ← [qc_generate] ← [qc_generate] ← [qc_map_flow]
--	--

```

├── specs/
│   ├── {PBI_ID}/
│   │   ├── spec.md          ← INPUT bắt buộc cho qc_generate
│   │   ├── plan.md         ← INPUT bắt buộc cho qc_generate
│   │   ├── assets/*.png     ← INPUT optional (UI mockup)
│   │   ├── testcase.md      ← [qc_generate] sinh; [qc_auto_test] cập nhật
│   │   └── auto_test_results/
│   │       ├── traces/
│   │       │   ├── TC-{NNN}.zip      ← [qc_auto_test] trace per-TC
│   │       │   └── screenshots/
│   │       │       ├── TC-{NNN}-fail-*.png ← [qc_auto_test] ảnh khi FAILED / auth expired
│   │       │       ├── run-{DATETIME}-summary.md ← [qc_auto_test] Data Chaining log + Component mới
│   │       │       ├── HISTORY.md      ← [qc_auto_test] lịch sử chạy (Engine: cli+mcp)
│   │       └── triage-{PBI_ID}-{YYYYMMDD}.md ← [qc_triage]
│   └── memory/
│       ├── constitution.md          ← INPUT (quy tắc dự án)
│       ├── domain-knowledge.md      ← INPUT (business domain)
│       ├── flow-index.md            ← [qc_user_flow] index thời gian + trạng thái
│       ├── userflow-{group}.md      ← [qc_user_flow] ASCII diagram + mô tả
│       ├── sub-system-map.json      ← [qc_sub-system-map]
│       └── components/
│           └── component_{alias}.md ← [qc_detect_component] per màn hình
├── rules/
│   └── component-rule.md            ← [qc_component_rule] selector rules
└── templates/
    └── testcase_template.md        ← INPUT template cho qc_generate (optional)

```

Yêu cầu

- Claude Code CLI (phiên bản hỗ trợ **claude plugin**)
- Node.js >= 16
- Project có Playwright (@playwright/test)
- File **.specify/tests/.env** với **ENV=<tên_env>**
- File **.specify/tests/.env.{ENV}** với **APP_URL=http://...**
- File **.specify/tests/user.json** (sinh tự động khi chạy **global-setup.ts** lần đầu)

Quy ước

- Skill **không tự xóa** dữ liệu cũ.
- Skill **không thay đổi** **.env** nếu file đã tồn tại.
- **qc_pre** tự gọi **qc_setup** và **qc_basepage** — không cần gọi riêng khi init project.
- **qc_basepage** và **qc_setup** là internal — không chạy tự động trong pipeline ngoài **qc_pre**.
- TC đã **✅ PASSED** / **❌ FAILED** sẽ **không chạy lại** ở lần sau (trừ khi dùng **--reset**).
- TC **⏸ SKIPPED** / **🟪 Chưa test** sẽ **chạy lại** lần tiếp theo.
- **playwright-cli** = mọi thao tác execution; **playwright-mcp** = chỉ discovery DOM (selector mới + Kendo combobox).
- **datafake.json** lưu tại **.specify/tests/playwright/{PBI_ID}/** (cùng thư mục spec.ts).
- **playwright.config.ts** phải có **testDir: './playwright'** và **tsconfig.json** có **baseUrl: ''**.
- Ưu tiên loại trừ INFRA **trước** khi kết luận APP_BUG trong qc_triage.

Thứ tự chạy đề xuất (project mới):

```

qc_pre → qc_detect_component → qc_component_rule → qc_user_flow
        → qc_map_flow → qc_url_page_map → qc_sub-system-map
        → qc_generate <PBI_ID> → qc_auto_test <PBI_ID> → qc_triage <PBI_ID>

```

Tài liệu chi tiết (format file, ví dụ đầy đủ, locator strategy): xem [VNR-QCKIT-FLOW.md](#)

Cập nhật: 2026-06-15